ЗВІТ

про виконання лабораторної роботи №6

«Робота з різними типами даних та моделями та відображення списками»

з дисципліни

«Програмування мобільних додатків»

Студента групи КН-2327Б

Багрія-Белза Андрія

Завдання: 1. Продовжити роботу над проєктом, розпочатим у Лабораторній роботі №5.

2. Створити окремий файл моделі (наприклад, item_model.dart).
3. Описати class Model з мінімум 4 полями: int, string, bool, enum.
4. Створити List<Model> — основне джерело даних екрану (мінімум 5 початкових елементів).
5. Відобразити список з ListView.builder та ListTile.
6. Реалізувати додавання, видалення елемента.
7. Реалізувати зміну стану елемента (іконка «обране» перемикає isFavorite; зміна значення status (enum) кнопкою/довгим натисканням; оновлення через setState()).
8. Створити Map<String, int> для статистики (загальна кількість елементів; кількість обраних; кількість виконаних).

1 Написання коду-розв’язку завдання:

```
// === lib/main.dart ===
import 'package:flutter/material.dart';
import 'screens/lab6_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Менеджер завдань — Лабораторна №6',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFFD4FF00),
          brightness: Brightness.dark,
          primary: const Color(0xFFD4FF00),
          surface: const Color(0xFF15181F),
        ),
        scaffoldBackgroundColor: const Color(0xFF0D0F13),
      ),
      home: const Lab6Screen(),
    );
  }
}

// === lib/models/task_model.dart ===
/// Перелік можливих статусів завдання (enum)
enum TaskStatus {
  active,
  done,
}

/// Модель завдання (Model class) із 4 обов'язковими типами полів:
/// - int (ідентифікатор)
```

```
/// - String (назва завдання)
/// - bool (прапорець обраного)
/// - enum TaskStatus (поточний статус виконання)
class TaskModel {
  final int id;
  String title;
  bool isFavorite;
  TaskStatus status;

  TaskModel({
    required this.id,
    required this.title,
    this.isFavorite = false,
    this.status = TaskStatus.active,
  });

  /// Зручний геттер для перевірки завершеності
  bool get isDone => status == TaskStatus.done;

  /// Метод для копіювання з модифікацією (за потреби)
  TaskModel copyWith({
    int? id,
    String? title,
    bool? isFavorite,
    TaskStatus? status,
  }) {
    return TaskModel(
      id: id ?? this.id,
      title: title ?? this.title,
      isFavorite: isFavorite ?? this.isFavorite,
      status: status ?? this.status,
    );
  }
}

/// Початковий список завдань (мінімум 5 елементів згідно з ТЗ)
List<TaskModel> getInitialTasks() {
  return [
    TaskModel(
      id: 1,
      title: 'Оптимізація запитів до бази даних',
      isFavorite: true,
      status: TaskStatus.active,
    ),
    TaskModel(
      id: 2,
      title: 'Рефакторинг архітектури Flutter-екранів',
      isFavorite: false,
      status: TaskStatus.done,
    ),
    TaskModel(
      id: 3,
      title: 'Інтеграція REST API та обробка помилок',
      isFavorite: false,
      status: TaskStatus.active,
    ),
    TaskModel(
      id: 4,
      title: 'Написання віджет-тестів та перевірка стану',
      isFavorite: true,
      status: TaskStatus.done,
    ),
    TaskModel(
      id: 5,
      title: 'Оновлення темної палітри кольорів Material 3',
      isFavorite: false,
      status: TaskStatus.active,
    ),
    TaskModel(
      id: 6,
      title: 'Підготовка збірки та публікація в Play Store',
      isFavorite: false,
      status: TaskStatus.active,
    ),
  ];
}

// === lib/screens/lab6_screen.dart (фрагмент) ===
class _Lab6ScreenState extends State<Lab6Screen> {
```

```
// Основне джерело даних екрана: List<TaskModel> (п. 4 Т3)
late List<TaskModel> _tasks;

// Поточний фільтр для зручності перегляду
TaskFilter _currentFilter = TaskFilter.all;

@override
void initState() {
  super.initState();
  // Ініціалізація мінімум 5 елементів
  _tasks = getInitialTasks();
}

// Обчислення статистики у вигляді Map<String, int> (п. 8 Т3)
Map<String, int> get _statistics => {
  'total': _tasks.length,
  'favorites': _tasks.where((t) => t.isFavorite).length,
  'done': _tasks.where((t) => t.status == TaskStatus.done).length,
  'active': _tasks.where((t) => t.status == TaskStatus.active).length,
};

// Фільтрований список завдань
List<TaskModel> get _filteredTasks {
  switch (_currentFilter) {
    case TaskFilter.active:
      return _tasks.where((t) => t.status == TaskStatus.active).toList();
    case TaskFilter.done:
      return _tasks.where((t) => t.status == TaskStatus.done).toList();
    case TaskFilter.favorites:
      return _tasks.where((t) => t.isFavorite).toList();
    case TaskFilter.all:
      return _tasks;
  }
}

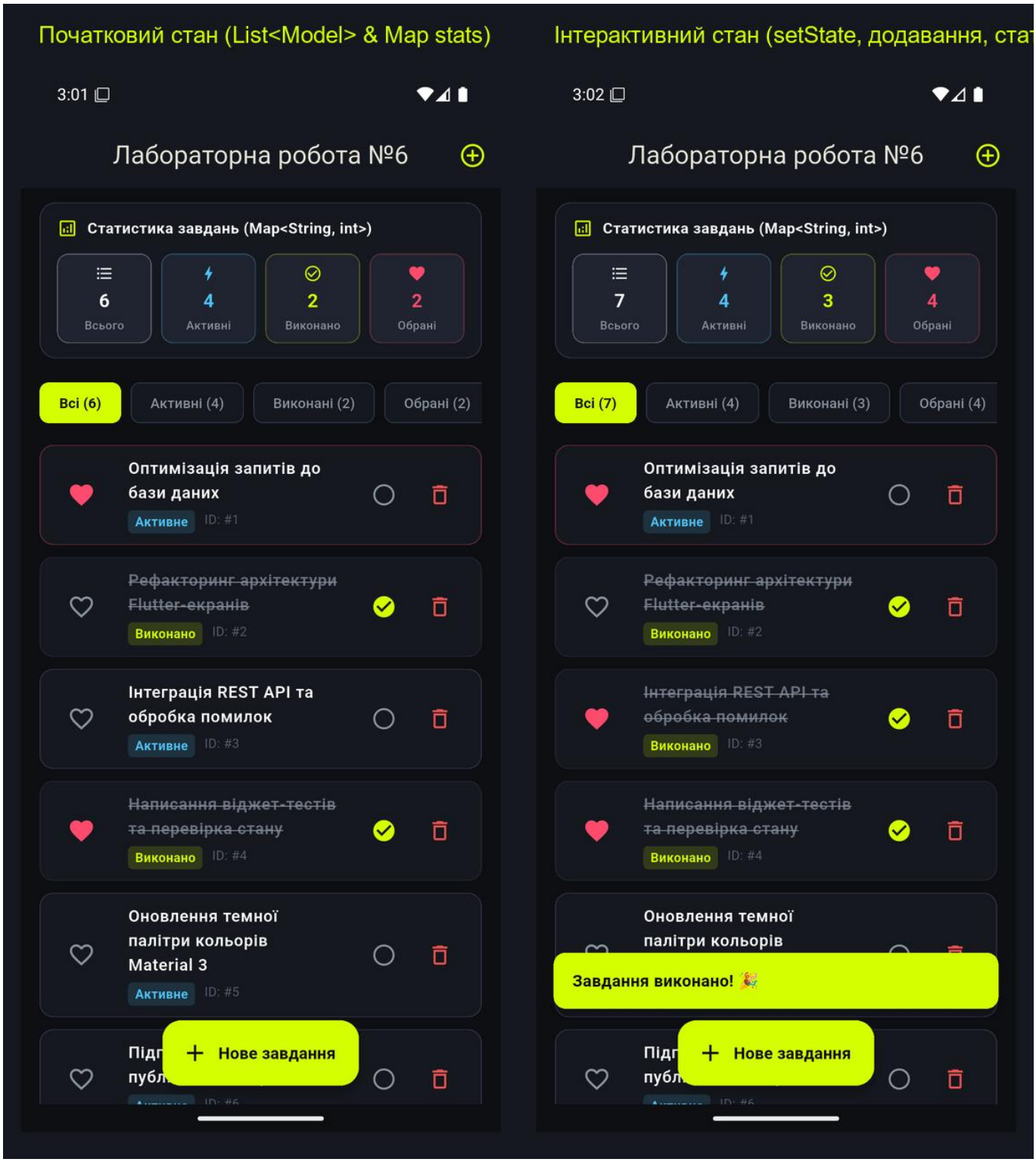
// Зміна обраного стану (п. 7 Т3)
void _toggleFavorite(TaskModel task) {
  setState(() {
    task.isFavorite = !task.isFavorite;
  });
  _showSnackBar(
    task.isFavorite ? 'Додано до обраного ❤️' : 'Видалено з обраного 🤍',
  );
}

// Зміна статусу enum (п. 7 Т3)
void _toggleStatus(TaskModel task) {
  setState(() {
    task.status = task.status == TaskStatus.done
        ? TaskStatus.active
        : TaskStatus.done;
  });
  _showSnackBar(
    task.status == TaskStatus.done
        ? 'Завдання виконано! 🎉'
        : 'Завдання позначено як активне',
  );
}
// ... (пропуск неістотних рядків)
@override
Widget build(BuildContext context) {
  final filtered = _filteredTasks;

  return Scaffold(
// ... (пропуск неістотних рядків)
    body: SafeArea(
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
// ... (пропуск неістотних рядків)
            Expanded(
              child: filtered.isEmpty
// ... (пропуск неістотних рядків)
                  : ListView.builder(
                      padding: const EdgeInsets.only(bottom: 80),
                      itemCount: filtered.length,
                      itemBuilder: (context, index) {
```

```
                        final task = filtered[index];
                        return TaskListItem(
                          task: task,
                          onToggleFavorite: () => _toggleFavorite(task),
                          onToggleStatus: () => _toggleStatus(task),
                          onDelete: () => _deleteTask(task),
                        );
                      },
                    ),
              ),
            ],
  // ... (пропуск неістотних рядків)
}
```

2 Результат виконання коду:



1. Чим List відрізняється від Map у Dart? List — це впорядкована індексована колекція елементів, доступ до яких здійснюється за цілочисельним індексом

(починаючи з 0), де елементи можуть дублюватися. Map — це асоціативна колекція пар «ключ — значення», де кожен ключ є унікальним, а доступ до значень виконується за відповідним ключем довільного типу (найчастіше String), що робить Map зручною структурою для словників, конфігурацій та агрегованої статистики.

2. Для чого використовують enum? Наведіть приклад. Переліки (enum) використовують для визначення фіксованого, безпечного з точки зору типізації набору іменованих значень чи станів. Це запобігає друкарським помилкам, характерним для рядкових констант, та забезпечує вичерпну перевірку в операторах switch. Приклад у проєкті: enum TaskStatus { active, done }, де кожен об'єкт завдання може набувати лише одного зі суворо визначених статусів.

3. Що таке модель даних (model class) і яку роль вона відіграє у Flutter-застосунку? Модель даних (Model Class) — це типізований користувацький клас Dart, що інкапсулює поля та логіку сутності предметної області (наприклад, TaskModel). Вона відокремлює шар бізнес-даних від віджетів інтерфейсу (UI), гарантує сувору типізацію, спрощує серіалізацію/десеріалізацію, покращує читабельність коду та полегшує тестування.

4. Як додати елемент у List<Model> та оновити інтерфейс? Для додавання елемента викликається метод .add() (або .insert()) на екземплярі List<Model>. Щоб Flutter перемалював екран і відобразив новий рядок у списку, цю операцію огортають у виклик setState() всередині класу стану StatefulWidget: setState(() { _tasks.add(newTask); });. Це сигналізує фреймворку про необхідність повторного виклику build().

5. Яка різниця між методами map() і where() для списку? Метод where() виконує фільтрацію елементів за предикатом (bool Function(E)), повертаючи лише ті елементи, які задовольняють умову (наприклад, _tasks.where((t) => t.isFavorite)). Метод map() здійснює трансформацію/проекцію: він викликає функцію перетворення для кожного елемента й повертає нову колекцію змінених значень (наприклад, _tasks.map((t) => t.title)). Обидва методи повертають лінивий Iterable.

6. Як реалізувати фільтрацію списку (наприклад, показувати лише обрані елементи)? Фільтрація реалізується за допомогою виклику методу .where() на основному списку: List<TaskModel> get filteredTasks => _tasks.where((t) => t.isFavorite).toList();. У ListView.builder передають довжину отриманого відфільтрованого списку в itemCount, а в itemBuilder будують елементи за індексом з filteredTasks[index].

7. Для чого використовується віджет ListView.builder? У чому його перевага? ListView.builder використовується для відображення динамічних або потенційно великих прокручуваних списків. Його ключова перевага полягає в оптимізації продуктивності та пам'яті через лінивий рендеринг (lazy rendering): itemBuilder генерує віджети рядків виключно тоді, коли вони потрапляють у видиму область екрана (viewport), не створюючи заздалегідь весь список у пам'яті.

8. Що таке itemCount, itemBuilder та index у ListView.builder? itemCount — це цілочисельний параметр, який визначає загальну кількість елементів у списку.

itemBuilder — функція зворотного виклику типу Widget Function(BuildContext, int), яка викликається рушієм рендерингу для побудови віджета конкретного рядка. index — порядковий номер (від 0 до itemCount - 1) поточного елемента, що рендериться у списку.

9. Як змінити стан окремого елемента списку та оновити UI? Для зміни стану окремого елемента списку знаходять відповідний об'єкт моделі (за індексом index або id), змінюють значення потрібного поля (наприклад, task.isFavorite = !task.isFavorite або task.status = TaskStatus.done) та викликають setState(). Flutter ініціює перезапуск методу build(), викликаючи itemBuilder для видимих рядків і оновлюючи їхнє графічне відображення.

10. Що станеться, якщо змінити дані моделі без виклику setState()? Змінна та відповідне поле об'єкта в оперативній пам'яті змінять своє значення, але фреймворк Flutter не отримає сповіщення про зміну стану. Екран не запланує перемалювання кадру, і користувацький інтерфейс залишиться зі застарілими даними доти, доки віджет випадково не перебудується внаслідок інших подій.

Висновок: На цій лабораторній роботі, я опанував практичні навички роботи зі структурами даних List та Map у мові Dart, створив типізовану модель даних TaskModel з обов'язковими полями різних типів (int, String, bool, enum TaskStatus), реалізував динамічне відображення колекції об'єктів за допомогою віджетів ListView.builder та ListTile, впровадив додавання нових завдань через модальний діалог і вилучення елементів, налаштував зміну внутрішнього стану моделей через setState() (перемикання прапорця обраного та зміну enum-статусу кліком і довгим натисканням), реалізував підрахунок та візуалізацію аналітичної статистики у вигляді Map<String, int>, а також верифікував коректність інтерфейсу та бізнес-логіки автоматизованими тестами й на емуляторі Android.